

# Konstruktör och destruktör

Varje klassdefinition har i normala fall en (eller flera) metod som fungerar som *konstruktörer*. En konstruktör är en speciell metod som exekveras då ett objekt skapas, och är menad att ta hand om diverse initialiseringar och parametrar. Normalt används konstruktörer för att ge initiala värden på diverse medlemmar i en klass. Klassen `Coordinate` har en konstruktör definierad på följande sätt:

```
Coordinate (float X, float Y) { m_X = X; m_Y = Y; };
```

Det som skiljer en konstruktördefinition från en vanlig metoddefinition är att den saknar returvärde och har *exakt* samma namn som klassen själv. Då vi skapar ett objekt med t.ex. följande anrop:

```
Coordinate C = Coordinate ( 19.8, 34.11 );
```

Är det konstruktorn som körs. Denna tar två parametrar som sätts som initialvärden för våra medlemmar. Vi kunde även definiera den på följande sätt:

```
Coordinate () { m_X = 0; m_Y = 0; };
```

varvid vi helt enkelt nollställer medlemmarna då klassen instantieras. Vi kan då inte använda samma syntax för att skapa objekt, utan nu måste vi manuellt sätta medlemmarna till önskade värden:

```
Coordinate C = Coordinate;  
C.setX ( 19.8 );  
C.setY ( 34.11 );
```

Notera att man kan placera en tom parentes efter `Coordinate C = Coordinate;` om man vill visa att konstruktorn inte tar några parametrar överhuvudtaget, men det är inte nödvändigt. Vilket sätt skall man då använda, eller kan man kanske använda båda sätten?

## Initialisering av medlemmar

I våra exempel ovan har konstruktorn egentligen inträffat något mera än kopierat givna parametrar till datamedlemmar. Kopierandet kan göras lättare om man vill. Istället för att i konstruktorns kropp explicit göra ett antal tilldelningar kan de göras direkt vid konstruktördefinitionen. Vi kan skriva om konstruktörerna från ovan till:

```
Coordinate (float X, float Y) : m_X(X), m_Y(Y) { }  
Coordinate () : m_X(0), m_Y(0) { }
```

Man placerar alltså direkt efter parentesen ett `:` (kolon) och räknar därefter upp de medlemmar som skall initialiseras och inom parentese de värden de skall få. Den första ger värden från parametrar, medan den andra ger hårdkodade värden. Har man flera initialiseringar placerar man ett kommatecken emellan. Efter initialiseringarna kommer det som normalt skulle komma. I vårt fall behöver vi inte längre någon kodkropp för konstruktorn, eftersom allt arbete redan gjorts.

Initialiseringarna görs *före* den kod som finns i konstruktorn exekveras, så själva kodkroppen kan använda sig av de initialiserade medlemmarna.

## Multipla konstruktörer

För att göra klasser lättare och bekvämare att använda kan man låta dem ha multipla konstruktörer. Meningen är att de skall ta olika parametrar (om alls) så att de kan skapas på olika sätt. Vi kan modifiera vår klass så att den tillåter båda konstruktörerna vi hittills sett:

```
class Coordinate {
public:
    // konstruktörer
    Coordinate (float X, float Y) { m_X = X; m_Y = Y; };
    Coordinate () { m_X = 0; m_Y = 0; };

    // accessera medlemmar
    float x () const { return m_X; };
    float y () const { return m_Y; };
    void setX (float X) { m_X = X; };
    void setY (float Y) { m_Y = Y; };

private:
    // medlemmar
    float m_X;
    float m_Y;
};
```

Nu kan man enkelt skapa objekt av klassen `Coordinate`. Man kan ha hur många konstruktörer som helst, så länge som ett visst set av parametrar endast kan användas av en konstruktor. De måste alltså vara unika. Om det finns två konstruktörer som tar likadana parametrar vet kompilatorn inte vilken som skall väljas och man får ett felmeddelande vid kompileringsskedet.

Om man inte definierar någon konstruktor överhuvudtaget i en klass kommer kompilatorn att skapa en tom konstruktor som inte gör någonting. Detta endast för att en konstruktor *måste* finnas. En tom konstruktor för `Coordinate` skulle se ut på detta sätt:

```
Coordinate () { };
```

Det är dock bättre att själv definiera denna konstruktor så att man kan vara säker på att medlemmar initialiseras till giltiga initialvärden.

## Copy-konstruktor

Det finns en annan typ av konstruktor som används förutom de som visats ovan. En speciell typ av konstruktor används vid vissa specialfall då objekt kopieras. Denna konstruktor kallas för (svengelskt sagt) en *copy-konstruktor*. Även detta är en metod som kompilatorn automatiskt genererar om den inte definieras av programmeraren. Denna konstruktor används bl.a. i följande situationer:

- då ett nytt objekt skapas som en kopia av ett annat redan existerande.
- då ett objekt skickas som parameter till en funktion eller metod. Gäller dock ej om det är frågan om en pekare eller referensparameter.

Allmänt ser en copy-konstruktor ut på detta sätt:

```
class namn {
    namn (const namn & variabel);
```

```
};
```

Den tar alltså som parameter ett objekt av samma typ som den själv. Objektet är definierat som en konstant parameter (`const`) som inte kan modifieras samt som en referensparameter för att göra anropet effektivare. Vad denna konstruktor skall göra är att den skall kopiera alla värden i *variabel* till "sig själv", alltså generera en kopia av denna. Copy-konstruktorn för vår klass kunde se ut på följande sätt:

```
Coordinate (const Coordinate & Original) { m_X = Original.x (); m_Y = Original.y
```

Här kopierar vi helt enkelt värdena som `Original` innehåller. Notera att eftersom `Original` är definierad som ett `const` objekt *kan man inte* anropa metoder för detta objekt som *inte* är definierade som `const`. Detta är orsaken till att vi tidigare definierade `x()` och `y()` som `const`-metoder. Några exempel på situationer då en copy-konstruktor används:

```
Coordinate C1;
// copy-konstruktorer
Coordinate C2 ( C1 );
Coordinate C3 = C1;
Coordinate C3 = Coordinate ( C1 );
```

Medan anropet:

```
Coordinate C1 (10, 10);
```

*inte* anropar någon copy-konstruktor. Det är alltså frågan om en ganska viktig konstruktor som det i många fall kan vara värt att definiera manuellt och inte lita på den definition som kompilatorn automatiskt ger åt oss. Kompilatorn kommer att kopiera data medlem för medlem från originalet till kopian, vilket ju nog var exakt samma sak som vi gjorde. Vi kunde alltså använt den automatiska definitionen i vårt fall, men i vissa fall kan man få allvarliga fel om man gör detta. Vi kommer att stöta på sådana klasser senare.

Copy-konstruktorn används även då man skickar objekt som värdeparametrar. Ett exempel:

```
#include <iostream>

class Coordinate {
public:
    // konstruktorer
    Coordinate (float X, float Y) { m_X = X; m_Y = Y; };
    Coordinate (const Coordinate & Original) { m_X = Original.x (); m_Y = Original.y (); };
    Coordinate () { m_X = 0; m_Y = 0; };

    // accessera medlemmar
    float x () const { return m_X; };
    float y () const { return m_Y; };
    void setX (float X) { m_X = X; };
    void setY (float Y) { m_Y = Y; };

private:
    // medlemmar
    float m_X;
    float m_Y;
};

// skriv ut en koordinat
void printCoordinate (Coordinate C) {
```

```

    cout << "X = " << C.x () << ", Y= " << C.y () << endl;
}

int main () {
    Coordinate C1 ( 1, 2);
    Coordinate C2 ( C1 );
    Coordinate C3 = C1;

    // skriv ut de olika koordinaterna
    printCoordinate ( C1 );
    printCoordinate ( C2 );
    printCoordinate ( C3 );
}

```

Vi har här en funktion `printCoordinate()` som tar som parameter ett objekt av typen `Coordinate`. Eftersom det skickas som *värdeparameter* kommer copy-konstruktorn att användas då den lokala variabeln `C` i funktionen `printCoordinate()` skapas. Copy-konstruktorn används alltså endast då objekt skickas som värdeparametrar. Om vi istället skirver om `printCoordinate()` till följande kommer vi inte att använda en copy-konstruktör för att skapa `C`:

```

void printCoordinate (Coordinate & C) {
    cout << "X = " << C.x () << ", Y= " << C.y () << endl;
}

```

Vi har nu undvikit en onödig konstruktion av variabeln `C` och även gjort vårt program en aning effektivare. Men i och med att vi skickar parametrarna till `printCoordinate()` som referensparametrar och vi inte har något behov av att ändra på dessa kan vi använda definitionen:

```

void printCoordinate (const Coordinate & C) {
    cout << "X = " << C.x () << ", Y= " << C.y () << endl;
}

```

Vi lägger alltså till definitionen `const` för parameter `Coordinate & C` för att tydligt markera att denna funktion inte modifierar referensparametern (även om den skulle kunna göra det). Det är god sed att använda `const` för parametrar, funktioner och metoder där det är lämpligt för att göra gränssnitt lättare att använda smat tydligare markera ut var objekt kan tänkas bli modifierade.

## Destruktor

Vi har hittills sett olika metoder för hur objekt kan skapas och hur konstruktörer används. I C++ finns det även en metod som körs då ett objekt förstörs, nämligen en *destruktör*. Denna exekveras för varje objekt vid en tidpunkt där objektet redan är "för sent att rädda". Destruktörer skall användas för att göra diverse uppstädning efter ett objekt. Det kan vara frågan om att frigöra allokerat minne, stänga öppnade filer eller frigöra andra resurser. I många fall måste destruktörer användas för att undvika minnesläckor. En destruktör definieras allmänt på detta sätt:

```

class namn {
    ...
    ~namn ();
    ...
};

```

Man har alltså en metod med samma namn som klassen själv, men som har ett `~` framför namnet. En destruktör tar aldrig parametrar och returnerar aldrig något. Då en destruktör är exekverad finns objektet inte längre. Man behöver aldrig manuellt kalla på en destruktör, det görs totalt automatiskt

vid rätt ögonblick. En generell regel är att ett objekt förstörs då det går "utanför scope", d.v.s. då det inte längre är aktuellt. I följande exempel förstörs alla de lokala objekten av typ `Coordinate` då funktionen de är definierade i avslutas:

```
void foo (Coordinate C) {
    Coordinate Tmp1, Tmp2;
    ...
}

int main () {
    Coordinate Point;
    ...
    foo ( Point );
    ...
}
```

De lokala variablerna `Tmp1`, `Tmp2` och `Point` förstörs då funktionerna avslutas, likaså parametern `C` som skapas i `foo()`. `Tmp1` och `Tmp2` skapas på nytt varje gång `foo()` anropas. En destruktör för vår `Coordinate`-klass har egentligen ingenting egentligen att göra, men vi kan skapa en tom destruktör:

```
class Coordinate {
public:
    // konstruktörer
    Coordinate (float X, float Y) { m_X = X; m_Y = Y; };
    Coordinate (const Coordinate & Original) { m_X = Original.x (); m_Y = Original.
    Coordinate () { m_X = 0; m_Y = 0; };

    // destruktör
    ~Coordinate () { };
    ...
};
```

Destruktorn gör ingenting. Vi kan istället se på en enkel vektor vars längd man kan dynamiskt specificera då den skapas. Vektorn har en konstruktor, en destruktör, två metoder för att läsa och skriva ett värde samt en för att returnera storleken.

```
#include <iostream>

class Vector {
public:
    // konstruktor och destruktör
    Vector (unsigned int Size) { m_Data = new int [Size]; m_Size = Size; }
    ~Vector () { }

    // accessmetoder
    void set (unsigned int Index, int Value) { m_Data[Index] = Value; }
    int get (unsigned int Index) const { return m_Data[Index]; }

    // storleken på vektorn
    unsigned int size () const { return m_Size; }

private:
    // pekare till datablock
    int * m_Data;

    // storlek på vektor
    unsigned int m_Size;
};
```

```
int main () {

    // skapa en dynamisk vektor med 100 element
    Vector V ( 100 );

    // fyll den med data
    for ( int Index = 0; Index < 100; Index++ ) {
        V.set ( Index, Index * 10 );
    }
}
```

I konstruktorn allokeras nu ett minnesblock som rymmer `Size` element. Den observante märker att programmet är mycket dåligt skrivet, vi gör inga kontroller överhuvudtaget att givna `Index` är inom rimliga gränser, men det duger för vårt exempel. Tittar man litet närmare på programmet ser man förstås att det allokerar minne med `new` i konstruktorn, men aldrig frigör minnet! Vi har även en tom destruktör som inte gör någonting överhuvudtaget. Hur skall vi kunna undvika att läcka minne om vi skapar och förstör upprepade instanser av `Vector`? Jo, vi kan se till att frigöra minnet i destruktorn. Det är precis dylika uppgifter som skall skötas i en destruktör. Vår nya destruktör blir således:

```
~Vector () { delete [] m_Data; }
```

Här borde även göras kontroller på att `m_Data` verkligen är allokerad, men det lämnas som en övning för läsaren. Nu läcker vår klass inte längre minne.

## Mera om copy-konstruktör

Vi har nu fått ett annat problem med vår `Vector`-klass. Vad händer om vi kopierar ett objekt till ett annat, t.ex. på följande sätt:

```
Vector Source ( 10 );
Vector Destination ( Source );
```

Kör vi detta program kommer vi att få en *segmentation fault*, som indikerar att vi gör något fel ved vår minneshantering. Vad går fel och varför? den automatiska copy-konstruktör som kompilatorn skapar oss kopierar alla medlemmar skilt för sig, d.v.s. `m_Size` och `m_Data` kopieras. Dock kopieras endast pekarens värde, inte minnesområdet som pekaren pekar på. Vi får således två pekare som pekar på samma minnesområde, vilket inte i sig är ett problem. Problemet framkommer då objekten förstörs. Eftersom båda pekar på samma område frigör det ena objektet minnet i destruktorn precis som det är menat, men då det andra objektet försöker frigöra samma minne är det reda frigjort och ett fel uppstår. Man kan komma undan detta genom att skapa en copy-konstruktör som ser till att kopiera även det minne som pekaren pekar på, så att det nya objektet pekar på ett eget minnesområde. Vi definieraren copy-konstruktör den enligt följande:

```
class Vector {
...
    Vector (const Vector & Original) {
        // allokeras minne
        m_Size = Original.size ();
        m_Data = new int [ m_Size ];

        // kopiera minnet
        memcpy ( m_Data, Original.m_Data, sizeof(int) * m_Size );
    }
}
```

Funktionen `memcpy()` som användes för att utföra kopieringen är en funktion som är inkluderad i

och med `#include <iostream>`. Den kopierar data från ett ställe till ett annat, och ersätter i vårt fall en `for`-slinga. Nu har klassen inte längre samma problem med minne som frigörs två gånger. Inte heller läcker det minne. Nu återstår endast felhantering för att göra den komplett.

Den observante märkte säkert att den nya copy-konstruktor kunde accessera den privata datamedlemmen `m_Data` i `Original`. Hur kan detta gå för sig, den borde väl vara skyddad? Jo, den är skyddad, men endast för objekt av andra klasser, inte för objekt av samma klass! Detta kan vara bra att veta då man t.ex. skriver copy-konstruktorer eller andra metoder som tar en (eller flera) parameter av samma klass.

---

[Förutgående](#)

En första klass

[Hem](#)[Upp](#)[Nästa](#)

Nästlade klasser